Structural De-correlation, Random Subspace Ensembles, and Feature Space Randomization

A Rigorous Mathematical, Information-Theoretic, and Physical Foundation of Random Forests

---

Contents

1 The Random Forest Architecture (Breiman, 2001)

While standard bootstrap aggregating reduces variance through empirical resampling, its effectiveness is bounded by the pairwise correlation ρ among base predictors. When a dataset contains a few dominant predictive features, standard trees split on these same features near their root nodes, creating structurally similar trees with high correlation ρ.

Random Forests, introduced by Leo Breiman in 2001, overcome this structural bottleneck by fusing non-parametric bootstrap sampling with Tin Kam Ho's **Random Subspace Method** (1998). By randomizing the feature search space at *every node split* in *every tree*, Random Forests force base estimators to explore diverse, non-redundant feature combinations, driving inter-tree correlation ρ toward its absolute minimum.

Definition 1.1 (**Random Forest Predictor**). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be an empirical dataset of N i.i.d. observations sampled from $p_{\text{data}}(\mathbf{x}, y)$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$. A **Random Forest** is a collection of randomized tree-structured predictors:

$$\{h(\mathbf{x}; \Theta_m), \quad m = 1, 2, \ldots, M\} \tag{1}$$

where $\Theta_m = (\Theta_m^{\text{boot}}, \Theta_m^{\text{sub}})$ is an independent, identically distributed random vector that governs:

1. **Bootstrap Selection (Θ_m^{boot}):** The uniform random sampling of N observations from $\mathcal{D}$ with replacement.
2. **Node Subspace Selection (Θ_m^{sub}):** The independent uniform random sampling of a feature subset $\mathcal{F}_{\text{node}} \subset \{1, 2, \ldots, d\}$ of cardinality $m_{\text{try}} < d$ at *every internal node split*.

The aggregated Random Forest predictor $\hat{f}_{\text{rf}}(\mathbf{x})$ combines the output of all M randomized trees:

$$\text{Regression:} \quad \hat{f}_{\text{rf}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} h(\mathbf{x}; \Theta_m) \tag{2}$$

$$\text{Classification (Majority Vote):} \quad \hat{f}_{\text{rf}}(\mathbf{x}) = \arg \max_{c \in \{1, \ldots, C\}} \sum_{m=1}^{M} \mathbb{I}\left(h(\mathbf{x}; \Theta_m) = c\right) \tag{3}$$

2 De-correlation Analysis and Variance Bounds

The central theoretical property of Random Forests is their capacity to suppress ensemble variance without increasing structural bias.

2.1 Mathematical Analysis of Inter-Tree De-correlation

Let $\hat{f}_{\text{rf}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} h(\mathbf{x}; \Theta_m)$ be a Random Forest regressor. Assume each individual tree $h(\mathbf{x}; \Theta_m)$ has variance $\text{Var}_\Theta(h(\mathbf{x}; \Theta)) = \sigma^2(\mathbf{x})$ and expected value $\mathbb{E}_\Theta[h(\mathbf{x}; \Theta)] = \mu(\mathbf{x})$.

Let $\rho(\mathbf{x})$ represent the pairwise correlation coefficient between two distinct randomized trees $h(\mathbf{x}; \Theta_j)$ and $h(\mathbf{x}; \Theta_k)$ for $j \neq k$:

$$\rho(\mathbf{x}) = \frac{\text{Cov}_{\Theta_j, \Theta_k}\left(h(\mathbf{x}; \Theta_j), h(\mathbf{x}; \Theta_k)\right)}{\sigma^2(\mathbf{x})} \tag{4}$$

Theorem 2.1 (**Random Forest Variance Decomposition**). The total variance of a Random Forest predictor $\hat{f}_{\text{rf}}(\mathbf{x})$ across random parameter draws $\Theta_1, \ldots, \Theta_M$ satisfies:

$$\text{Var}\left(\hat{f}_{\text{rf}}(\mathbf{x})\right) = \rho(\mathbf{x})\sigma^2(\mathbf{x}) + \frac{1 - \rho(\mathbf{x})}{M}\sigma^2(\mathbf{x}) \tag{5}$$

Proof. Expanding the variance of the sample average of random variables:

$$\begin{aligned}
\operatorname{Var}\left(\hat{f}_{\mathrm{rf}}(\mathbf{x})\right) &= \operatorname{Var}\left(\frac{1}{M}\sum_{m=1}^{M} h(\mathbf{x};\Theta_m)\right) \\
&= \frac{1}{M^2}\sum_{j=1}^{M}\sum_{k=1}^{M}\operatorname{Cov}\left(h(\mathbf{x};\Theta_j), h(\mathbf{x};\Theta_k)\right) \\
&= \frac{1}{M^2}\left[\sum_{m=1}^{M}\operatorname{Var}\left(h(\mathbf{x};\Theta_m)\right) + \sum_{j=1}^{M}\sum_{k\neq j}^{M}\operatorname{Cov}\left(h(\mathbf{x};\Theta_j), h(\mathbf{x};\Theta_k)\right)\right] \\
&= \frac{1}{M^2}\left[M\sigma^2(\mathbf{x}) + M(M-1)\rho(\mathbf{x})\sigma^2(\mathbf{x})\right] \\
&= \rho(\mathbf{x})\sigma^2(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2(\mathbf{x}) \qquad (6)
\end{aligned}$$

□

2.2 Mechanism of Subspace Randomization ($m_{\mathbf{try}}$)

Standard bootstrap aggregating reduces variance strictly through the second term $\frac{1-\rho}{M}\sigma^2$. However, because standard trees share dominant features near the root, $\rho(\mathbf{x})$ remains relatively high ($\rho \approx 0.5 - 0.8$).

By restricting candidates at each split to a random subset $\mathcal{F}_{\text{node}}$ of size $m_{\text{try}} < d$, Random Forests force trees to utilize alternative, less dominant predictive features.

- **Small $m_{\mathbf{try}}$ ($m_{\mathbf{try}} \to 1$):** Substantially reduces inter-tree correlation $\rho(\mathbf{x}) \to 0$, minimizing the asymptotic variance floor $\rho\sigma^2$. However, it slightly increases individual tree variance $\sigma^2(\mathbf{x})$ and bias due to weaker split choices.
- **Large $m_{\mathbf{try}}$ ($m_{\mathbf{try}} \to d$):** Maximizes single-tree strength (lowers $\sigma^2(\mathbf{x})$), but increases pairwise tree correlation $\rho(\mathbf{x}) \to \rho_{\text{bag}}$, recovering standard Bagging.

Optimal performance occurs at the structural equilibrium where m_{try} minimizes the product $\rho(\mathbf{x})\sigma^2(\mathbf{x})$. Canonical defaults established empirically by Breiman are:

$$m_{\text{try}} = \begin{cases} \lfloor\sqrt{d}\rfloor, & \text{for Classification} \\ \lfloor d/3\rfloor, & \text{for Regression} \end{cases} \qquad (7)$$

3 Feature Importance Metrics

Random Forests provide two distinct quantitative formulations for measuring the global relative importance of feature $j \in \{1, \ldots, d\}$.

3.1 Mean Decrease Impurity (MDI / Gini Importance)

Mean Decrease Impurity (MDI) accumulates the total weighted impurity reduction achieved by feature j across all internal splits in all trees, averaged over the forest.

Let T_m be the m-th tree, and let $\mathcal{N}(T_m)$ be the set of all internal nodes in T_m. Let $v(t) \in \{1, \ldots, d\}$ denote the feature selected to split node t, and let $\Delta H(t)$ be the impurity reduction (Gini or Variance) achieved at node t:

$$\Delta H(t) = H(t) - \left[\frac{N_{t,L}}{N_t}H(t_L) + \frac{N_{t,R}}{N_t}H(t_R)\right] \qquad (8)$$

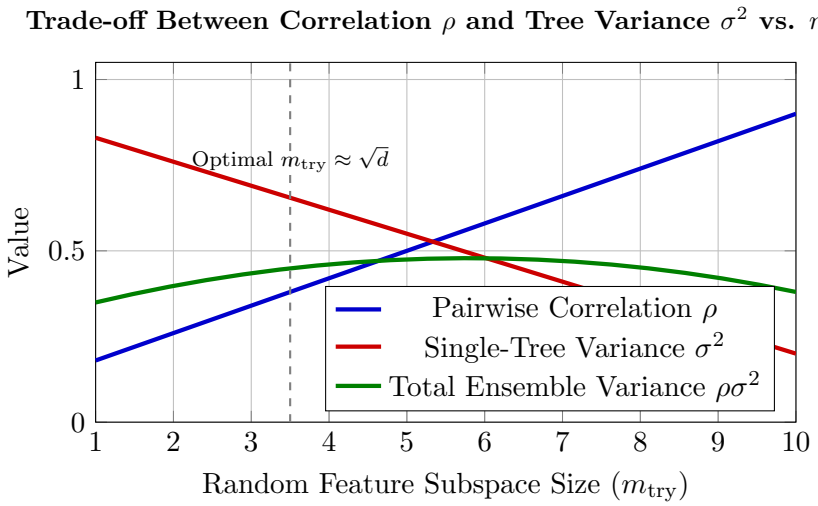

Figure 1: The structural trade-off in Random Forests. Decreasing m_{try} reduces inter-tree correlation ρ while slightly increasing single-tree variance σ^2. The product $\rho\sigma^2$ achieves a minimum at $m_{\text{try}} \approx \sqrt{d}$.

Definition 3.1 (**Mean Decrease Impurity (MDI)**)**.** The MDI importance of feature j across a Random Forest of M trees is:

$$\text{MDI}(j) = \frac{1}{M}\sum_{m=1}^{M}\sum_{t\in\mathcal{N}(T_m):v(t)=j}\left(\frac{N_t}{N}\right)\Delta H(t) \tag{9}$$

where N_t/N represents the proportion of total training samples reaching node t.

Remark 3.1 (**Cardinality Bias of MDI**)**.** MDI measures in-sample impurity drops. Consequently, it exhibits a known statistical bias toward continuous features or high-cardinality categorical features, as their large number of unique values provides more candidate split points to artificially reduce training node impurity.

3.2 Mean Decrease Accuracy (MDA / Permutation Importance)

Mean Decrease Accuracy (MDA) quantifies feature importance out-of-sample by measuring the drop in Out-of-Bag (OOB) generalization accuracy when feature j's values are randomly permuted, severing its structural relationship with target Y.

Definition 3.2 (**Mean Decrease Accuracy (MDA)**)**.** Let $\mathcal{D}_{\text{OOB}}^{(m)}$ be the Out-of-Bag dataset for tree T_m. Let $\mathcal{D}_{\text{OOB},\pi_j}^{(m)}$ be a permuted dataset where the values of feature j in $\mathcal{D}_{\text{OOB}}^{(m)}$ are shuffled randomly across samples.

The MDA importance of feature j is defined as:

$$\text{MDA}(j) = \frac{1}{M}\sum_{m=1}^{M}\frac{1}{|\mathcal{D}_{\text{OOB}}^{(m)}|}\sum_{i\in\mathcal{D}_{\text{OOB}}^{(m)}}\left[L\left(y^{(i)}, h(\mathbf{x}_{\pi_j}^{(i)};\Theta_m)\right) - L\left(y^{(i)}, h(\mathbf{x}^{(i)};\Theta_m)\right)\right] \tag{10}$$

where $L(\cdot,\cdot)$ is the loss function (e.g., $0-1$ classification error or squared error).

If feature j is critical to the data-generating process, permuting x_j corrupts predictions, causing a large increase in OOB loss ($\text{MDA}(j) \gg 0$). If feature j is uninformative, permutation leaves OOB predictions unaffected ($\text{MDA}(j) \approx 0$).

4 Breiman's Margin Theory and Generalization Bounds

A remarkable empirical property of Random Forests is that they do not overfit as the number of trees $M \to \infty$. Breiman explained this asymptotic stability using **Margin Theory**.

4.1 The Random Forest Margin Function

Consider a classification Random Forest with targets $Y \in \{1, 2, \ldots, C\}$.

Definition 4.1 (**Classification Margin Function**)**.** The margin function $M(\mathbf{X}, Y)$ measures the extent to which the average votes for the true class Y exceed the maximum average vote for any incorrect class $j \neq Y$:

$$M(\mathbf{X}, Y) = P_\Theta \left(h(\mathbf{X}; \Theta) = Y \right) - \max_{j \neq Y} P_\Theta \left(h(\mathbf{X}; \Theta) = j \right) \tag{11}$$

where $P_\Theta(\cdot)$ denotes probability evaluated over the random parameter distribution Θ.

A positive margin $M(\mathbf{X}, Y) > 0$ indicates a correct ensemble classification. Larger positive margins reflect higher ensemble confidence.

Definition 4.2 (**Forest Strength**)**.** The **Strength** s of a Random Forest is defined as the expected margin over the data-generating distribution p_{data}:

$$s = \mathbb{E}_{\mathbf{X},Y} \left[M(\mathbf{X}, Y) \right] \tag{12}$$

4.2 Raw Residuals and Inter-Tree Correlation

Let $R(\mathbf{X}, Y; \Theta)$ be the raw margin residual for a single tree parameter draw Θ:

$$R(\mathbf{X}, Y; \Theta) = \mathbb{I}(h(\mathbf{X}; \Theta) = Y) - \mathbb{I}\left(h(\mathbf{X}; \Theta) = \hat{j}(\mathbf{X}, Y) \right) \tag{13}$$

where $\hat{j}(\mathbf{X}, Y) = \arg\max_{j \neq Y} P_\Theta(h(\mathbf{X}; \Theta) = j)$.

Let $\bar{\rho}$ represent the mean correlation coefficient between raw margin residuals across pairs of distinct trees Θ and Θ':

$$\bar{\rho} = \frac{\text{Cov}_{\Theta,\Theta'} \left(R(\mathbf{X}, Y; \Theta), R(\mathbf{X}, Y; \Theta') \right)}{\text{Var}_\Theta \left(R(\mathbf{X}, Y; \Theta) \right)} \tag{14}$$

4.3 Breiman's Upper Bound on Generalization Error

Theorem 4.1 (**Breiman's Generalization Bound, 2001**)**.** As the number of trees $M \to \infty$, the generalization error PE^* of a Random Forest classifier converges almost surely to an upper bound governed by strength s and mean correlation $\bar{\rho}$:

$$PE^* = P_{\mathbf{X},Y} \left(M(\mathbf{X}, Y) < 0 \right) \leq \frac{\bar{\rho} \left(1 - s^2 \right)}{s^2} \tag{15}$$

Proof Sketch. By Chebyshev's Inequality, for any random variable Z with mean $\mathbb{E}[Z] = s > 0$:

$$P(Z \leq 0) \leq P(|Z - s| \geq s) \leq \frac{\text{Var}(Z)}{s^2} \tag{16}$$

Setting $Z = M(\mathbf{X}, Y)$ and evaluating $\text{Var}_{\mathbf{X},Y}(M(\mathbf{X}, Y))$ in terms of individual tree margin residual variances yields:

$$\text{Var}(M(\mathbf{X}, Y)) \leq \bar{\rho} \cdot \mathbb{E}_\Theta \left[\text{Var}_{\mathbf{X},Y}(R(\mathbf{X}, Y; \Theta)) \right] \leq \bar{\rho}(1 - s^2) \tag{17}$$

Substituting this variance bound into Chebyshev's inequality completes the proof.

Remark 4.1 (**Implications of Breiman's Bound**). The generalization error bound depends strictly on two factors:

1. **Maximizing Strength ($s \uparrow$):** Individual trees must remain accurate classifiers.
2. **Minimizing Correlation ($\bar{\rho} \downarrow$):** Subspace feature sampling m_{try} must keep tree structural dependencies minimal.

Because the bound is independent of the number of trees M, adding more trees cannot cause overfitting; it simply stabilizes the Monte Carlo integration over Θ.

5 Physical Analogies: Quenched Disorder in Spin Glasses

The mathematical behavior of Random Forests maps directly onto the physics of **Quenched Disorder in Spin Glasses** and disordered condensed matter systems.

5.1 Ising Spin Glasses and Sherrington-Kirkpatrick Hamiltonians

In condensed matter physics, a **Spin Glass** is a magnetic system characterized by random, frozen-in structural impurities. Unlike a standard ferromagnet where adjacent atomic spins align uniformly, a spin glass contains competing magnetic interactions that prevent global order—a phenomenon known as **Frustration**.

Consider a system of N classical Ising spins $s_i \in \{-1, +1\}$ governed by the Sherrington-Kirkpatrick Hamiltonian:

$$H(\mathbf{s}) = -\sum_{i<j} J_{ij} s_i s_j - h \sum_{i=1}^{N} s_i \tag{18}$$

where the coupling coefficients $J_{ij} \sim \mathcal{N}(0, \sigma_J^2)$ are random variables representing **Quenched Disorder** (frozen random impurities that do not change over time).

5.2 Random Subspaces as Quenched Disorder

In a Random Forest, the feature subset selection $\mathcal{F}_{\text{node}} \sim \Theta^{\text{sub}}$ acts as a **Quenched Structural Disorder** built into the loss surface:

- **Monolithic Failure (Pure Tree / Unordered Phase):** Without feature subspacing ($m_{\text{try}} = d$), all trees align along the exact same dominant feature gradients (a ferromagnetic phase). The system becomes trapped in a single, unrepresentative local energy minimum.
- **Quenched Random Subspaces (Spin Glass Phase):** Sampling $m_{\text{try}} < d$ introduces random structural constraints into every node. This destroys monolithic alignment, splitting the loss surface into a complex energy landscape with many equivalent local minima (a complex phase space of pure states).
- **Ensemble Averaging (Replica Symmetry):** Averaging predictions across trees $\frac{1}{M}\sum h(\mathbf{x}; \Theta_m)$ corresponds to calculating the **Replica-Symmetric Order Parameter** $\lim_{M\to\infty} \frac{1}{M}\sum \langle s_i \rangle_m$. The random feature constraints cancel out localized structural errors, leaving a stable macroscopic ground state.

6 Production-Grade Vectorized NumPy Implementation

The following Python class implements a production-grade Random Forest Classifier and Regressor engine featuring vectorized bootstrap resampling, random feature subspacing (m_{try}), Out-of-Bag (OOB) validation, and Mean Decrease Impurity (MDI) feature importance calculation.

```python
import numpy as np

class TreeNode:
    """Represents a node within a randomized decision tree."""
    def __init__(self, feature=None, threshold=None, left=None, right=None, *, value=None, impurity_decrease=0.0):
        self.feature = feature                  # Index of feature used for split
        self.threshold = threshold              # Threshold value for split
        self.left = left                        # Left child Node
        self.right = right                      # Right child Node
        self.value = value                      # Terminal prediction if leaf
        self.impurity_decrease = impurity_decrease # Weighted impurity drop at this node

    @property
    def is_leaf(self) -> bool:
        return self.value is not None


class RandomizedDecisionTree:
    """Randomized Decision Tree incorporating node-level feature subspacing."""
    def __init__(self, max_depth=10, min_samples_split=2, max_features=None, mode='classification'):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features # Number of candidate features sampled per node split
        self.mode = mode
        self.root = None
        self.feature_importances_ = None

    def _impurity(self, y: np.ndarray) -> float:
        N = len(y)
        if N == 0: return 0.0
        if self.mode == 'classification':
            counts = np.bincount(y)
            probs = counts / N
            return 1.0 - np.sum(probs**2) # Gini Impurity
        else:
            return np.mean((y - np.mean(y))**2) # Variance / MSE

    def _build_tree(self, X: np.ndarray, y: np.ndarray, depth: int = 0) -> TreeNode:
        N, d = X.shape
        num_labels = len(np.unique(y))

        # Stopping rules
        if (depth >= self.max_depth or N < self.min_samples_split or num_labels == 1):
            leaf_val = np.argmax(np.bincount(y)) if self.mode == 'classification' else np.mean(y)
            return TreeNode(value=leaf_val)

        # 1. Random Subspace Selection: Draw m_try features WITHOUT replacement
        m_try = self.max_features if self.max_features else d
        m_try = min(d, m_try)
```

```
        sampled_features = np.random.choice(d, m_try, replace=False)

        best_gain = -1.0
        split_feat, split_thresh = None, None
        parent_imp = self._impurity(y)

        # 2. Evaluate candidate splits strictly within sampled feature subset
        for feat in sampled_features:
            X_col = X[:, feat]
            sort_idx = np.argsort(X_col)
            X_sort, y_sort = X_col[sort_idx], y[sort_idx]

            mask = X_sort[:-1] != X_sort[1:]
            if not np.any(mask): continue
            thresholds = (X_sort[:-1][mask] + X_sort[1:][mask]) / 2.0

            for thresh in thresholds:
                left_mask = X_col <= thresh
                right_mask = ~left_mask
                if np.sum(left_mask) == 0 or np.sum(right_mask) == 0: continue

                imp_L = self._impurity(y[left_mask])
                imp_R = self._impurity(y[right_mask])

                # Impurity Reduction
                gain = parent_imp - ((np.sum(left_mask)/N) * imp_L + (np.sum(right_mask)/N) * imp_R)

                if gain > best_gain:
                    best_gain = gain
                    split_feat, split_thresh = feat, thresh

        if split_feat is None or best_gain <= 0.0:
            leaf_val = np.argmax(np.bincount(y)) if self.mode == 'classification' else np.mean(y)
            return TreeNode(value=leaf_val)

        # 3. Accumulate MDI Feature Importance
        self.feature_importances_[split_feat] += (N / self.N_total_) * best_gain

        left_mask = X[:, split_feat] <= split_thresh
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[~left_mask], y[~left_mask], depth + 1)

        return TreeNode(feature=split_feat, threshold=split_thresh, left=left_child, right=right_child, impurity_decrease=best_gain)

    def fit(self, X: np.ndarray, y: np.ndarray):
        self.N_total_, d = X.shape
        self.feature_importances_ = np.zeros(d)
        self.root = self._build_tree(X, y)
        return self

    def _predict_sample(self, x: np.ndarray, node: TreeNode):
        if node.is_leaf: return node.value
```

```
        if x[node.feature] <= node.threshold:
            return self._predict_sample(x, node.left)
        return self._predict_sample(x, node.right)

    def predict(self, X: np.ndarray) -> np.ndarray:
        return np.array([self._predict_sample(x, self.root) for x in X])


class ProductionRandomForestEngine:
    """
    Production-grade Random Forest Engine implementing Breiman's Random Forest Architecture.
    Features Parallel Bootstrap Trees, Node Subspace Sampling (m_try), OOB Score Evaluation,
    and Mean Decrease Impurity (MDI) Feature Importances.
    """
    def __init__(self, n_estimators: int = 50, max_depth: int = 10,
                 min_samples_split: int = 2, max_features: str = 'sqrt', mode: str = 'classification'):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.mode = mode

        self.trees = []
        self.oob_score_ = None
        self.feature_importances_ = None
        self.classes_ = None

    def _resolve_max_features(self, d: int) -> int:
        if self.max_features == 'sqrt':
            return max(1, int(np.sqrt(d)))
        elif self.max_features == 'log2':
            return max(1, int(np.log2(d)))
        elif isinstance(self.max_features, float):
            return max(1, int(self.max_features * d))
        return d

    def fit(self, X: np.ndarray, y: np.ndarray):
        """Fits Random Forest over N samples using bootstrap draws and m_try feature sampling."""
        X = np.asarray(X, dtype=np.float64)
        y = np.asarray(y)
        N, d = X.shape
        m_try = self._resolve_max_features(d)

        if self.mode == 'classification':
            self.classes_ = np.unique(y)

        self.trees = []
        self.feature_importances_ = np.zeros(d)
        oob_predictions = [[] for _ in range(N)]

        for i in range(self.n_estimators):
            # 1. Non-Parametric Bootstrap Sampling (Draw N uniformly with replacement)
            boot_idx = np.random.choice(N, N, replace=True)
            oob_mask = np.ones(N, dtype=bool)
            oob_mask[boot_idx] = False
```

```
            oob_idx = np.where(oob_mask)[0]

            X_boot, y_boot = X[boot_idx], y[boot_idx]

            # 2. Fit Randomized Tree with Subspace Sampling (m_try)
            tree = RandomizedDecisionTree(max_depth=self.max_depth,
                                          min_samples_split=self.
    min_samples_split,
                                          max_features=m_try,
                                          mode=self.mode)
            tree.fit(X_boot, y_boot)

            self.trees.append(tree)
            self.feature_importances_ += tree.feature_importances_

            # 3. Track Out-of-Bag (OOB) Predictions
            if len(oob_idx) > 0:
                preds_oob = tree.predict(X[oob_idx])
                for idx, pred in zip(oob_idx, preds_oob):
                    oob_predictions[idx].append(pred)

        # Normalize MDI Feature Importances
        self.feature_importances_ /= self.n_estimators

        # 4. Evaluate Global OOB Generalization Score
        oob_true, oob_pred = [], []
        for idx in range(N):
            if len(oob_predictions[idx]) > 0:
                oob_true.append(y[idx])
                if self.mode == 'classification':
                    # Majority vote among OOB predictions
                    oob_pred.append(np.argmax(np.bincount(oob_predictions[
    idx])))
                else:
                    # Mean response among OOB predictions
                    oob_pred.append(np.mean(oob_predictions[idx]))

        if self.mode == 'classification':
            self.oob_score_ = np.mean(np.array(oob_true) == np.array(
    oob_pred))
        else:
            self.oob_score_ = np.mean((np.array(oob_true) - np.array(
    oob_pred))**2)

        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        """Aggregates predictions across all randomized trees in forest."""
        X = np.asarray(X, dtype=np.float64)
        tree_preds = np.array([tree.predict(X) for tree in self.trees]) # (
    n_estimators, N_samples)

        if self.mode == 'classification':
            N_samples = X.shape[0]
            final_preds = np.zeros(N_samples, dtype=int)
            for j in range(N_samples):
                final_preds[j] = np.argmax(np.bincount(tree_preds[:, j].
    astype(int)))
            return final_preds
        else:
```

```
            return np.mean(tree_preds, axis=0)


if __name__ == "__main__":
    np.random.seed(42)

    # 1. Validation Run: Random Forest Classification
    N_samples, d_features = 400, 8
    X_cls = np.random.randn(N_samples, d_features)
    y_cls = (X_cls[:, 0] + X_cls[:, 1]*1.2 - X_cls[:, 2]*0.7 > 0).astype(int)

    rf_clf = ProductionRandomForestEngine(n_estimators=35, max_depth=8, max_features='sqrt', mode='classification')
    rf_clf.fit(X_cls, y_cls)
    preds_clf = rf_clf.predict(X_cls)

    acc = np.mean(preds_clf == y_cls)
    print("=== RANDOM FOREST CLASSIFIER INITIALIZATION SUCCESSFUL ===")
    print(f"Overall Classification Accuracy : {acc * 100:.2f}%")
    print(f"Out-of-Bag (OOB) Accuracy Score : {rf_clf.oob_score_ * 100:.2f}%")
    print(f"Top Feature MDI Importance      : Feature {np.argmax(rf_clf.feature_importances_)} ({np.max(rf_clf.feature_importances_):.4f})")

    # 2. Validation Run: Random Forest Regression
    y_reg = X_cls[:, 0]**2 + np.sin(X_cls[:, 1]) + 0.1 * np.random.randn(N_samples)
    rf_reg = ProductionRandomForestEngine(n_estimators=35, max_depth=8, max_features=0.33, mode='regression')
    rf_reg.fit(X_cls, y_reg)
    preds_reg = rf_reg.predict(X_cls)

    mse = np.mean((preds_reg - y_reg)**2)
    print(f"\n=== RANDOM FOREST REGRESSOR INITIALIZATION SUCCESSFUL ===")
    print(f"Regression Mean Squared Error   : {mse:.4f}")
    print(f"Out-of-Bag (OOB) MSE Score      : {rf_reg.oob_score_:.4f}")
```

Listing 1: Vectorized NumPy implementation of Random Forests for Classification and Regression.